

RAG Pipeline offloading via NVMe-oF for Edge LLM Inference



202255570 양윤성 yyouseong@pusan.ac.kr

202455325 나예은 nye6998@pusan.ac.kr

지도교수 안성용

목 차

1. 서론.....	1
1.1. 연구 배경.....	1
1.2. 기존 문제점.....	1
1.3. 연구 목표.....	2
2. 문제 정의 및 해결 방안.....	3
2.1. 문제 정의.....	3
2.1.1. 엣지 환경의 저장 공간 제약.....	3
2.1.2. Vector DB 접근 시 I/O 병목.....	3
2.1.3. 기존 접근 방식의 한계.....	3
2.2. 해결 방안.....	4
2.2.1. 분리형 Storage 구조.....	4
2.2.2. RAG 연산을 Storage Server로 이동.....	4
2.2.3. NVME-oF 기반 RAG 및 Tokenization 오프로딩 구조.....	4
3. 연구 내용.....	5
3.1. 설계 목표와 시스템 전체 구조.....	5
3.1.1. 설계 목표.....	5
3.1.2. 시스템 전체 구조.....	6
3.1.3. 처리 위치의 분리와 구현 범위.....	7
3.1.4. 구성 요소별 변경과 역할.....	8
3.2. Edge Device의 오프로딩 요청과 추론 입력 연동.....	8
3.2.1. CLI 입력과 오프로딩 선택.....	9

3.2.2.	입력 파일과 결과 영역 준비.....	10
3.2.3.	NVMe 명령 구성과 전송.....	11
3.2.4.	결과 회수와 추론 입력 전달.....	12
3.3.	Storage Server의 명령 처리와 RAG 수행.....	13
3.3.1.	사용자 명령 등록과 요청 분기.....	13
3.3.2.	입력 데이터의 비동기 읽기.....	14
3.3.3.	RAG 파이프라인 수행.....	15
3.3.4.	토큰화 결과 기록과 요청 완료.....	16
3.4.	인터페이스 적용 조건과 구현상의 제약.....	17
3.4.1.	주소 매핑과 결과 형식의 유지 조건.....	17
3.4.2.	오류 처리와 요청 수명.....	18
3.4.3.	구현 의존성과 평가 범위.....	18
4.	연구 결과 분석 및 평가.....	19
4.1.	실험 환경.....	19
4.2.	실험 결과.....	20
4.2.1.	데이터 이동량 비교.....	20
4.2.2.	latency breakdown.....	20
4.2.3.	similarity search breakdown.....	21
5.	결론 및 향후 연구 방향.....	21
5.1.	결론.....	21
5.2.	향후 연구 방향.....	22
6.	참고 문헌.....	22

1. 서론

1.1. 연구 배경

최근 대규모 언어 모델(LLM, Large Language Model)은 다양한 분야에서 활용되고 있으나, 사실과 다른 내용을 생성하는 환각 현상과 최신 정보를 반영하기 어렵다는 한계가 있다. 검색 증강 생성(RAG, Retrieval-Augmented Generation)은 외부 문서나 데이터베이스에서 검색한 관련 정보로 모델의 입력을 보강하여 이러한 한계를 완화하는 기술이다. 이를 통해 모델을 재학습하지 않고도 최신 정보나 특정 분야의 지식을 응답 생성에 활용할 수 있다.

그러나 RAG에 사용하는 Vector DB의 규모가 커질수록 저장 공간과 데이터 접근에 대한 부담도 증가한다. 특히 자원이 제한된 Edge Device에서는 대규모 벡터 데이터와 인덱스를 로컬에 저장하고 관리하기 어려울 수 있다. 이러한 제약에 대응하는 방법으로, 컴퓨팅과 저장 공간을 분리하고 벡터 데이터와 인덱스를 원격 Storage Server에 배치하는 분리형 스토리지 아키텍처를 활용할 수 있다.

이러한 구성에서 RAG의 검색 연산을 Edge Device에서 수행하면, 검색 과정에서 원격 스토리지의 인덱스와 데이터에 반복적으로 접근하게 된다. 이때 네트워크를 통한 데이터 이동과 요청·응답에 따른 통신 지연이 누적되어 RAG 처리 성능에 영향을 줄 수 있다. 따라서 Edge 환경에서 RAG를 활용할 때에는 장치의 저장 용량뿐 아니라, 원격 데이터 접근에 따른 전송 부담과 처리 지연도 함께 고려해야 한다.

1.2. 기존 문제점

대규모 RAG 시스템을 분리형 스토리지 환경에서 운용할 때 나타나는 가장 큰 문제점은 원격 인덱스 접근에 따른 심각한 네트워크 I/O 병목과 통신 지연이다. 기존 방식에서는 Vector DB 인덱스만 원격 Storage Server에 두고 유사도 검색 연산을 Edge Device에서 수행한다. 이 경우 그래프 기반 벡터 인덱스(예: DiskANN)를 탐색하는 과정에서 수많은 노드와 벡터 블록을 읽기 위해 Edge와 스토리지 간 네트워크 패브릭을 반복적으로 왕복 순회하게 된다. 이로 인해 쿼리당 수백 밀리초 단위의 통신 지연이 누적되어 실시간 엣지 서비스의 응답 성능을 저해한다.

1.3. 연구 목표

본 연구의 목적은 자원이 제한된 Edge Device에서 대규모 RAG 데이터베이스를 효율적으로 활용하기 위해, NVMe over Fabrics(NVMe-oF) 기반의 RAG 파이프라인 오프로딩 구조를 설계하고 구현하는 것이다. 대규모 Vector DB를 원격 Storage Server에 저장하고 Edge에서 RAG를 수행하는 환경에서는, 검색 과정에서 원격 인덱스에 반복적으로 접근하여 네트워크를 통한 데이터 이동과 처리 지연이 발생한다. 이를 개선하기 위해 데이터가 저장된 Storage Server에서 RAG 파이프라인과 tokenization을 수행하고, Edge에서는 반환된 토큰 시퀀스를 이용해 LLM 추론을 수행하는 Near-Data Processing 구조를 적용하고자 한다. 이를 통해 원격 인덱스 접근에 따른 데이터 이동을 줄이고, Edge 환경에서 RAG 기반 LLM 서비스의 처리 효율을 높이는 것을 목표로 한다.

구체적인 연구 목적은 다음과 같다.

1. Edge와 Storage Server 간 RAG 파이프라인 오프로딩 구조 설계

질의 임베딩, 유사도 검색, 검색 결과를 반영한 프롬프트 구성 및 tokenization을 Storage Server로 오프로딩하고, Edge에서는 LLM 추론을 수행하도록 역할을 분리한다.

2. NVMe-oF 기반 오프로딩 경로 구현

NVMe-oF와 SPDK를 활용하여 Edge의 질의를 Storage Server에 전달하고, 서버에서 처리한 토큰 시퀀스를 Edge가 읽을 수 있도록 반환하는 사용자 정의 NVMe 명령 기반의 처리 경로를 구현한다.

3. Storage Server의 RAG 처리 결과와 Edge의 LLM 추론 연동

Storage Server에서 생성한 토큰 시퀀스를 Edge의 LLM 입력으로 직접 연결하여, 질의 전달부터 검색, tokenization, 응답 생성까지 이어지는 통합 시스템을 구현한다.

4. 데이터 이동량 및 처리 지연 감소 효과 검증

Edge에서 원격 인덱스에 접근하며 RAG를 수행하는 기존 방식과 제안 구조를 비교하여, Edge-Storage Server 간 데이터 이동량과 RAG 단계별 처리 지연을 측정하고 오프로딩의 효과를 평가한다.

2. 문제 정의 및 해결 방안

2.1. 문제 정의

2.1.1. 엣지 환경의 저장 공간 제약

언어 데이터의 집합인 Corpus의 크기가 증가할수록 Vector DB의 규모도 같이 증가하고, 전체 Vector Index를 메모리에 유지하는 것이 어려워진다. 특히 저장공간이 제한된 Edge Device의 경우 대규모 Vector DB를 로컬에 저장하면서 LLM inference까지 동시에 수행하기 어렵다. 따라서 Vector DB를 Edge가 아닌 별도의 Storage Server에 두는 구조가 필요하다. 그러나 저장공간 문제를 해결하기 위해 데이터를 원격으로 분리하면 Edge와 Storage Server 사이에서 데이터 이동이 발생한다.

2.1.2. Vector DB 접근 시 I/O 병목

Vector DB의 크기가 커지면 Host Memory와 Storage 사이의 데이터 이동 자체가 중요한 병목이 될 수 있다. 특히 Vector DB가 Storage Server에 있고 Similarity Search가 Edge에서 수행되는 구조에서는 검색 과정에서 Storage를 접근이 발생한다. 이 경우 저장 장치의 I/O와 네트워크 통신이 함께 발생하므로 추가적인 지연이 발생할 수 있다.

2.1.3. 기존 접근 방식의 한계

Near-data processing(NDP)은 데이터가 저장된 위치와 최대한 가까운 곳에서 연산을 수행하는 방식이다. RAG는 Similarity Search 과정에서 Storage의 Vector DB에 반복적으로 접근하므로 NDP를 적용하기에 적합하다.

그러나 위의 방식은 별도의 하드웨어를 필요로 하기 때문에 일반적인 Edge환경에서 재현하거나 적용하기 어렵다. 또한 Retrieval 단계만 Storage로 이동하고, Tokenization은 여전히 Edge에서 수행하므로, 검색된 Augmented Text를 다시 Edge로 전송하면서 데이터 이동이 발생한다.

2.2. 해결 방안

2.2.1. 분리형 Storage 구조

대규모 Corpus와 Vector DB는 Edge 내부에 저장하지 않고 별도의 Storage Server에 배치한다. Edge는 LLM inference에 필요한 기능을 중심으로 구성하고, 대규모 데이터와 검색 처리는 Storage Server가 담당하도록 하여 Edge의 저장공간 부담을 줄인다.

2.2.2. RAG 연산을 Storage Server로 이동

Embedding과 Similarity Search를 데이터가 위치한 Storage Server에서 수행한다. 이를 통해 Vector Index 탐색 과정에서 원격 Storage 접근이 네트워크를 반복적으로 통과하는 것을 줄여 네트워크 지연과 데이터 이동에 따른 오버헤드를 완화한다. 즉, Edge는 사용자의 질의를 Storage Server에 전달하고 처리된 결과를 전달받는 구조로 구성한다.

2.2.3. NVME-oF 기반 RAG 및 Tokenization 오프로딩 구조

Edge와 Storage Server 간의 오프로딩 요청은 NVMe-oF를 기반으로 구성하며, Custom NVMe Command를 통해 질의와 결과를 전달한다. Storage Server에서는 SPDK 기반 NVMe-oF Target에 Custom Command Handler를 추가하여 RAG와 Tokenization을 수행하고, 생성된 Token을 Edge로 전달한다. 이를 통해 Retrieval만 Storage에서 수행하고 Tokenization은 Edge에서 처리하는 기존 방식과 달리, Tokenization까지 Storage Server에서 수행하여 Augmented Text를 다시 Edge로 전송하는 데이터 이동을 줄이고자 한다.

3. 연구 내용

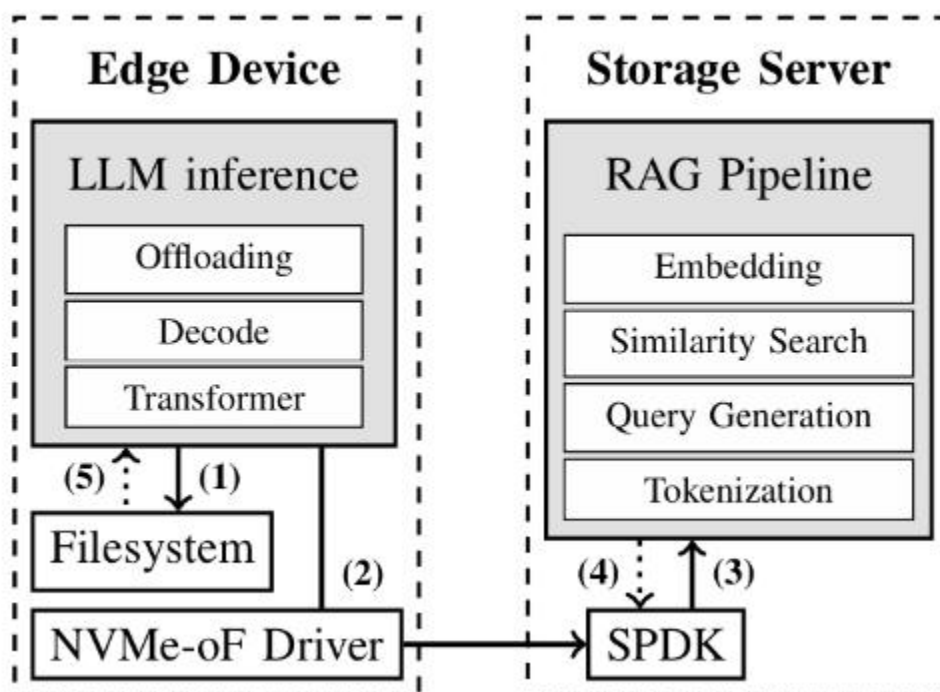
본 장에서는 NVMe-oF 기반의 분리형 스토리지 환경에서 RAG 파이프라인을 Storage Server로 오프로딩 하기 위해 구현한 시스템의 구조와 동작을 설명한다. 이를 위해 Edge의 LLM 실행 환경인 llama.cpp와 Storage Server의 NVMe-oF 처리를 담당하는 SPDK를 수정하고, 두 시스템이 하나의 RAG기반 LLM 서비스로 연동되도록 구성하였다. Edge는 사용자 대화를 모델 입력으로 구성하고 답변 추론을 수행하며, 오프로딩 경로에서는 Storage의 SPDK 애플리케이션이 검색을 위한 외부 서비스 호출, 문서 결합 및 토큰화를 담당한다. 두 실행 위치는 파일의 저장 구간을 기술한 메타데이터와 사용자 정의 NVMe 명령으로 연결된다.

3.1. 설계 목표와 시스템 전체 구조

3.1.1. 설계 목표

본 시스템은 앞서 정의한 Edge 환경의 저장공간 제약과 원격 DB 접근에 따른 I/O 오버헤드를 줄이는 것을 설계 목표로 하였다. 이를 위해 RAG 파이프라인과 tokenization 처리를 Storage Server로 오프로딩하고, Edge Device에서는 Storage Server에서 반환된 토큰을 이용하여 LLM 추론을 수행하도록 구성하였다. 이러한 구조는 검색과 입력 전처리를 데이터가 저장된 위치에 가까운 곳에서 수행하는 NDP(Near-Data Processing) 방식에 해당한다. 이를 통해 대규모 Vector DB를 Edge에 직접 저장하지 않으면서도 원격 데이터에 대한 반복적인 접근을 줄이고, Edge와 Storage Server 사이의 처리 역할을 분리하여 데이터 이동과 전처리 부담을 줄이는 것을 목표로 한다.

3.1.2. 시스템 전체 구조



사용자 입력과 답변 생성을 담당하는 Edge Device와 RAG 및 Tokenization을 담당하는 Storage Server로 구성되며, 두 구성 요소는 NVMe-oF를 통해 연결된다. Edge Device에는 llama.cpp 기반의 LLM 추론 기능, 파일시스템 및 NVMe-oF 드라이버가 위치한다. Storage Server에는 사용자 정의 NVMe 명령을 처리하는 SPDK 타킷과 RAG 파이프라인이 위치하며, SPDK는 SSD에 저장된 입력과 결과 영역에 접근한다.

Edge Device는 사용자 입력을 준비하고 스토리지 서버에 오프로딩을 요청하는 역할을 수행한다. 오프로딩 경로에서는 Edge의 RAG 및 Tokenization 처리를 Storage Server로 넘기고, 반환된 토큰 ID 배열을 llama.cpp의 기존 토큰 입력 경로에 전달한다. 따라서 Storage Server가 생성한 입력 토큰을 문자열로 변환하여 다시 Tokenization을 하지 않고 답변 생성에 사용할 수 있다.

Storage Server는 입력 질의의 임베딩 생성, 유사도 검색, 검색 문서와 질의의 결합 및 Tokenization을 수행한다. 임베딩 서비스에서 얻은 벡터를 이용하여 PostgreSQL 기반 Vector DB에 검색을 요청하고, 검색된 문서를 사용자 질의 및 이전 문맥과 결합하여 LLM 입력을 구성한다. 이후 결합된 입력을 Tokenization하고, 생성된 토큰 시퀀스를 Edge가 지정한 결과 영역에 기록한다. 그림의 Query Generation은 이러한 검색 문서와 질의의 결합 과정을 의미한다.

오프로딩 요청부터 답변 생성까지의 처리 순서는 다음과 같다.

1. Edge는 입력 프롬프트 파일과 결과를 저장할 파일을 준비하고, FIEMAP을 이용하여 두 파일의 저장 구간 정보를 조회한다.
2. Edge는 입력·결과 파일의 저장 구간을 나타내는 메타데이터를 사용자 정의 NVMe 명령에 담아 SPDK 타깃으로 전달한다.
3. Storage Server는 전달받은 저장 구간에서 입력을 읽고, RAG 파이프라인과 토큰화를 수행한다.
4. Storage Server는 결과 영역에 토큰 개수와 토큰 ID 배열을 기록하고, 쓰기가 완료되면 명령을 완료한다.
5. Edge는 Direct I/O로 결과 파일을 읽어 토큰 시퀀스를 회수하고, 이를 LLM 추론 입력으로 전달하여 답변을 생성한다.

이 구조는 검색과 Tokenization의 실행 위치를 Storage Server로 이동시키고, 처리 결과를 토큰 단위로 Edge의 추론 과정에 연결한다. 다만 오프로딩 과정에는 명령 메타데이터 전송뿐 아니라 입력 파일 쓰기와 결과 파일 읽기도 포함된다. 따라서 실제 데이터 이동량과 처리 지연의 변화는 이러한 경로를 포함한 실험을 통해 평가한다.

3.1.3. 처리 위치의 분리와 구현 범위

본 구현은 llama.cpp의 CLI 입력 경로에 RAG 전처리를 추가하고, 오프로딩 설정에 따라 전처리의 실행 위치를 선택하도록 구성하였다. 오프로딩을 사용하지 않는 경우에는 Edge에서 임베딩 서비스와 PostgreSQL에 요청하여 검색 문서를 결합하고, 기존 입력 처리 경로에서 Tokenization을 수행한다. 오프로딩을 사용하는 경우에는 Storage Server에서 RAG 전처리와 Tokenization을 수행하고, Edge가 반환된 토큰 ID 배열을 추론 입력으로 사용한다. 따라서 오프로딩을 사용하지 않는 상태도 RAG 전처리를 수행하는 경로에 해당한다.

두 경로는 전처리 결과의 표현에서도 차이가 있다. Edge 경로는 검색 문서가 결합된 문자열을 기존 입력 처리 경로에 전달하고, Storage 경로는 이미 Tokenization된 숫자 배열을 전달한다. 이 기능은 CLI 작업 경로를 대상으로 추가한 것으로, 일반 HTTP 요청 전체에 자동으로 적용되는 것은 아니다. 또한 반환된 토큰을 추론에 사용하려면 Storage Server의 Tokenization에 사용한 단어와 특수 토큰 정책이 Edge의 추론 모델과 일치해야 한다.

3.1.4. 구성 요소별 변경과 역할

구현에서 추가한 제어 경로를 구성 요소별로 정리한 것이다. llama.cpp에서는 CLI 작업에 오프로딩 선택과 측정값을 저장하고 파일 기반 요청 함수를 연결하였다. SPDK에서는 사용자 I/O 명령을 등록할 수 있는 경로와 별도 nvme_tokenize 애플리케이션을 추가하였다. HTTP, JSON, PostgreSQL 및 Tokenization 라이브러리의 기존 기능을 조합하며, 반입한 외부 헤더의 일반 기능을 새로 개발한 연구 기여로 포함하지 않는다.

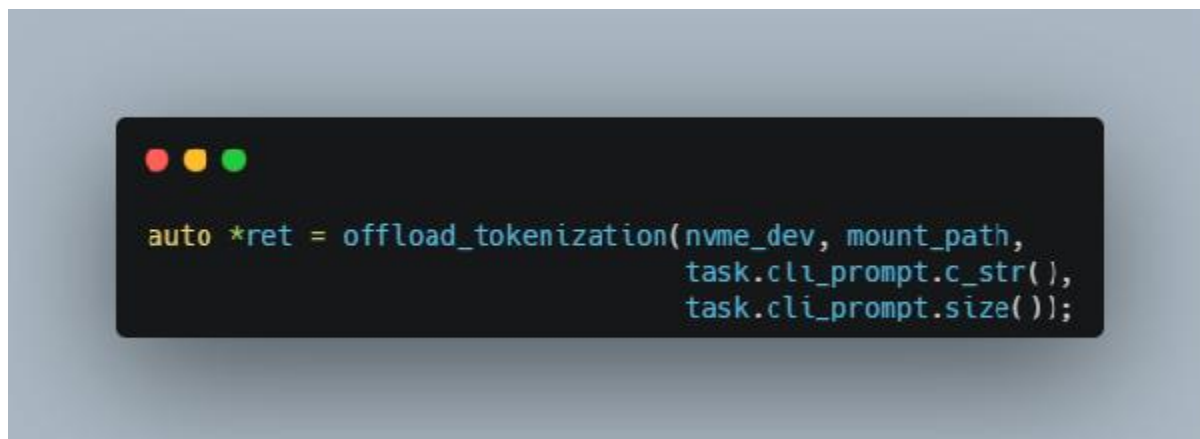
구성 요소	추가한 경로	시스템에서의 역할
llama.cpp CLI	작업별 오프로딩 선택과 시간 전달	실행 위치 선택과 추론 입력 연결
Edge offload 모듈	파일 준비·FIEMAP·NVMe 명령·결과 읽기	파일의 저장 구간과 결과 버퍼 관리
SPDK 명령 처리	사용자 opcode 등록·분기·방향 지정	NVMe-oF 요청을 새 처리기로 연결
Storage 애플리케이션	비동기 I/O·RAG 호출·토큰화	저장 구간 읽기부터 결과 기록까지 수행

3.2. Edge Device의 오프로딩 요청과 추론 입력 연동

Edge Device는 llama.cpp의 llama-cli를 통해 사용자의 질의를 입력받고, RAG 오프로딩이 활성화된 경우 입력 프롬프트 파일의 저장 구간을 사용자 정의 NVMe 명령으로 Storage Server에 전달한다. Storage Server에서 처리된 토큰 결과를 회수한 뒤, 이를 LLM 추론 입력으로 사용하여 최종 응답을 생성한다. 본 절에서는 Edge Device에서 수행되는 오프로딩 요청 준비, NVMe 명령 전송, 결과 회수 및 추론 연결까지의 전체 과정을 설명한다.

3.2.1. CLI 입력과 오프로딩 선택

사용자는 llama.cpp의 llama-cli를 통해 질의를 입력한다. CLI에는 /toggle-rag-offload 명령을 추가하여 RAG 오프로딩의 활성화 여부를 전환할 수 있도록 하였다. 선택값의 초기 상태는 비활성(false)이며, 이 값은 RAG 기능 자체의 비활성화가 아니라 Edge에서 RAG를 수행하는 경로를 의미한다. 오프로딩이 활성화되면 CLI 프롬프트는 offload_tokenization() 함수로 전달되어 Storage Server를 통한 처리 경로로 진입한다.



여기서 task.cli_prompt는 단일 질의 문자열만이 아니라 대화 템플릿과 이전 문맥이 포함될 수 있는 CLI 프롬프트이다. 장치 경로와 마운트 경로는 각각 LLAMA_NVME_DEVICE와 LLAMA_MOUNT_PATH 환경변수에서 읽는다. 새 사용자 요청을 만들 때 현재 토크 값을 작업 객체에 복사하므로, 이후 토크 변경이 이미 대기 중인 작업에 소급 적용되지는 않는다. 이 제어는 CLI 작업 경로에 추가한 기능이며 일반 HTTP 요청 전체에 자동 적용되지는 않는다.

3.2.2. 입력 파일과 결과 영역 준비

`offload_tokenization()`은 지정한 마운트 디렉터리에 `prompt.txt`와 `output.bin` 두 파일을 생성하거나 기존 파일을 잘라 다시 사용한다. 두 파일은 `O_DIRECT` 플래그로 열어 파일 데이터의 읽기/쓰기에서 Linux page cache를 우회하도록 한다. 파일 이름 생성과 extent 관리 자체는 Linux 파일시스템을 통해 이루어진다.

```
int input_fd = MUST_SUCCESS_POSIX(openat(mount_fd, "prompt.txt",
                                         O_WRONLY | O_DIRECT | O_CREAT | O_TRUNC, 0644));
int output_fd = MUST_SUCCESS_POSIX(openat(mount_fd, "output.bin",
                                           O_RDWR | O_DIRECT | O_CREAT | O_TRUNC, 0644));
```

`output.bin`은 결과를 받을 placeholder 파일이다. 현재 코드에서는 먼저 128 KiB의 정렬된 버퍼를 0으로 채워 결과 파일에 기록하고, 그 파일의 extent 정보를 조회한다. 이후 CLI 프롬프트를 `prompt.txt`에 기록하고 입력 파일의 extent 정보를 조회한다. 입력 버퍼의 주소는 `aligned_alloc(4096, ...)`으로 정렬하고, 기록할 크기는 4,096 byte의 배수로 올린다. 버퍼 전체를 0으로 초기화한 뒤 실제 프롬프트를 복사하여 남는 공간을 패딩으로 사용한다. 주소 정렬과 기록 크기 정렬은 서로 다른 조건이다.

각 파일을 기록한 뒤 `get_extent_info()`에서 Linux FIEMAP ioctl을 호출하여 파일시스템이 보고하는 물리적 저장 구간(extent)을 조회한다. 코드에서는 각 extent의 물리 위치와 길이를 512로 나누어 블록 시작 위치와 블록 수로 변환한다. 실제 호출은 위 순서대로 이루어지며, 아래는 두 조회 호출을 함께 발췌한 것이다.

```
struct extent_info *output_extent = get_extent_info(output_fd);
/* 입력 프롬프트를 기록한 뒤 */
struct extent_info *input_extent = get_extent_info(input_fd);
```

Storage Server는 파일명을 해석하는 대신 이 메타데이터를 bdev의 블록 주소로 사용한다. 이를 위해 파일시스템의 물리 주소와 대상 namespace의 bdev 주소가 같은 기준으로

대응해야 한다. 파티션 시작 위치와 장치 블록 크기를 포함한 주소 매핑 조건은 3.4.1절에서 설명한다.

3.2.3. NVMe 명령 구성과 전송

init_nvme_cmd()는 조회된 extent 정보를 바탕으로 사용자 정의 NVMe 명령을 구성한다. 명령의 데이터 영역에는 프롬프트 본문 전체가 아니라 입력 파일과 결과 파일의 extent 메타데이터를 담는다. 명령 구조체의 주요 필드는 다음과 같다.

```
*cmd = (struct nvme_passthru_cmd64) {
    .opcode = 0xD5,                // RAG 및 부달 규정 opcode
    .nsid   = 1,                  // 대상 namespace
    .addr   = (uint64_t)(uintptr_t)send_buf,
    .data_len = send_buf_size,
    .cdw10  = 0,                  // input metadata 시작 offset
    .cdw11  = input_bytes,        // output metadata 시작 offset
    .cdw12  = input->nr_extents,   // input extent 개수
    .cdw13  = output->nr_extents,  // output extent 개수
    .cdw14  = input->total_nr_blocks, // 총 input 블록 수
    .cdw15  = output->total_nr_blocks, // 총 output 블록 수
};
```

아래표는 각 명령 필드의 의미와 Edge에서 설정하는 값, Storage에서 해석하는 방법을 정리한 것이다.

필드	Edge의 값	Storage의 해석
opcode / nsid	0xD5 / 1	등록된 custom handler로 분기 / 대상 namespace
data_len	입력·출력 metadata 크기의 합(byte)	수신 요청의 실제 길이와 대조할 전송 크기
CDW10 / CDW11	0 / 입력 metadata byte 수	각 metadata 구조의 시작 offset (byte)
CDW12 / CDW13	입력 / 출력 extent 개수	metadata 배열 크기의 산정 기준
CDW14 / CDW15	입력 / 출력 총 블록 수	bdev block 크기를 곱해 byte로 변환

입력·출력 메타데이터는 각각 16 byte 헤더와 extent당 16 byte 항목으로 구성되며, 두 구조를 전송 버퍼에 연속하여 복사한다. `addr`는 Edge 프로세스의 전송 버퍼 주소이며 Storage의 LBA를 뜻하지 않는다. `nsid`는 현재 코드에서 1로 고정한다.

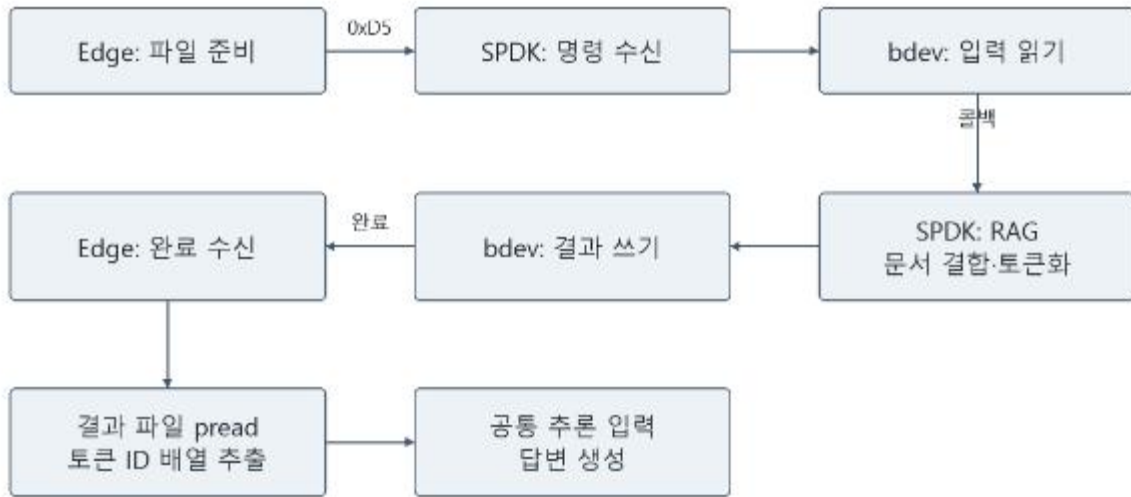
구성된 명령은 `nvme_submit_io_passthru64()`로 지정된 NVMe 장치에 제출한다. 이 장치가 NVMe-oF로 연결된 원격 namespace인 구성에서는 커널 드라이버를 거쳐 Storage Server의 SPDK NVMe-oF Target으로 전달된다. 이 호출은 동기 방식으로 동작하며, Storage Server에서 처리가 완료되어 NVMe completion이 반환될 때까지 Edge는 대기한다.

3.2.4. 결과 회수와 추론 입력 전달

정상 완료 경로에서 NVMe 명령의 동기 호출이 반환되면 Edge는 `output.bin`을 읽어 Storage Server가 기록한 토큰 결과를 회수한다. `o_DIRECT`로 열린 파일이므로 page cache를 우회하여 데이터를 읽는다. 다만 Direct I/O만으로 주소 매핑, 동시 접근 및 명령 오류를 포함한 전체 결과 일관성이 보장되는 것은 아니다.

결과 파일의 형식은 다음과 같다. Storage는 앞 4 byte에 `int32_t` 토큰 개수를 기록하고 Edge는 같은 영역을 `uint32_t nr_tokens`로 읽는다. 그 뒤에는 4 byte 단위의 토큰 ID가 연속하여 배치된다. 정상 결과의 토큰 개수는 음수가 아니어야 한다. 128 KiB 결과 공간에서 4 byte 개수 필드를 제외하면 32-bit 토큰은 최대 32,767개까지 저장할 수 있다. 이는 버퍼 크기로 계산한 용량 상한이며 측정된 처리량이 아니다.

회수한 토큰 ID는 `std::vector<uint32_t>`로 복사되어 `prompt`의 숫자 배열 입력으로 전달되며, 복사가 끝나면 `llama_tokenization_offload_free(ret)`로 반환 버퍼를 해제한다. 기존 입력 처리 함수는 숫자 배열을 토큰 배열로 받아들이므로, Storage Server에서 생성한 결과를 문자열로 되돌려 재토큰화하지 않는다. 이후 기존 모델 평가와 토큰 생성 루프를 통해 최종 응답을 생성한다. 그림의 Transformer와 Decode는 이 추론 과정을 개략적으로 나타내며, 코드에 두 개의 독립된 단계를 새로 추가했다는 의미는 아니다. 아래는 파일 준비부터 명령 완료와 결과 회수까지의 정상 처리 순서를 나타낸다.



3.3. Storage Server의 명령 처리와 RAG 수행

Storage Server는 SPDK 기반 NVMe-oF Target에 사용자 정의 I/O 명령 처리 기능을 추가하여, Edge Device로부터 전달된 오프로딩 요청을 수신하고 RAG 파이프라인과 토큰화를 수행한다. 본 절에서는 명령 등록 및 분기, 입력 데이터 읽기, RAG 파이프라인 수행, 토큰화 및 결과 기록까지의 전체 과정을 설명한다.

3.3.1. 사용자 명령 등록과 요청 분기

`nvmf_tokenize` 애플리케이션은 시작 시 `llama.cpp`의 모델 vocabulary를 `vocab_only` 모드로 로드하고, SPDK 서브시스템 초기화 이후 opcode `0xD5`에 대한 사용자 정의 I/O 명령 처리 함수를 등록한다.

```

spdk_nvmf_set_custom_io_cmd_hdr(0xD5,
                                &nvmf_bdev_ctrlr_tokenize_cmd,
                                SPDK_NVMF_DATA_HOST_TO_CONTROLLER);

```

이 등록은 opcode `0xD5`의 NVMe-oF 요청이 도착했을 때 해당 요청을 `nvmf_bdev_ctrlr_tokenize_cmd()` 함수로 분기하도록 SPDK의 I/O command dispatcher에

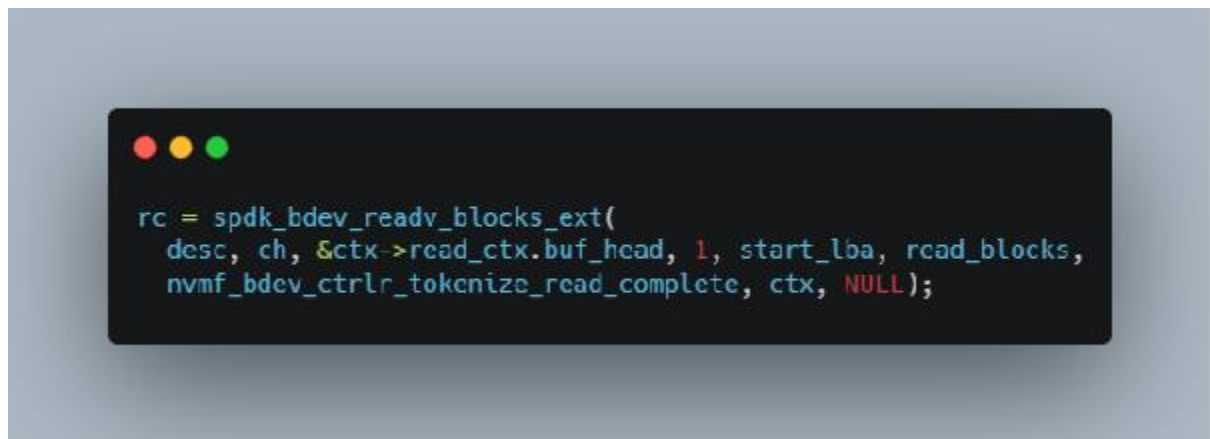
추가한다. 데이터 전송 방향은 HOST_TO_CONTROLLER로 지정하여 Edge가 보낸 extent 메타 데이터를 수신할 수 있도록 한다.

NVMe-oF 요청이 도착하면 SPDK의 `nvme_ctrlr_process_io_cmd()`에서 namespace, 접근 상태 등 기본 검사를 수행한 뒤, opcode를 기준으로 사용자 정의 I/O handler 테이블을 확인한다. 등록된 opcode가 발견되면 일반 passthrough 및 표준 I/O 명령 분기보다 먼저 사용자 정의 처리 함수를 호출한다.

3.3.2. 입력 데이터의 비동기 읽기

`nvme_bdev_ctrlr_tokenize_cmd()`는 Edge가 NVMe 명령의 데이터 영역에 담아 전달한 extent 메타데이터를 해석한다. CDW10~CDW15 필드와 수신된 데이터로부터 입력 파일(prompt.txt)과 결과 파일(output.bin)의 물리적 블록 위치, extent 개수, 총 블록 수를 추출한다.

각 extent가 bdev 범위에 포함되는지와 블록 수 합계를 확인한 뒤, 요청별 상태 구조체와 DMA 버퍼를 할당한다. 이후 입력 extent 정보를 이용하여 prompt.txt가 저장된 NVMe 블록 위치에서 입력 프롬프트를 읽어온다.



SPDK NVMe bdev로 SSD에 접근하는 구성에서는 이 읽기가 Linux 파일시스템의 page cache를 거치지 않는다. 이는 Edge의 파일 descriptor에 설정한 `O_DIRECT`와는 다른 I/O 경로이다. bdev의 실제 하위 장치 구성은 실행 환경에서 확인해야 한다. 입력 extent가 여러 개인 경우 각 read 완료 callback에서 다음 extent의 읽기를 순차적으로 제출하여 연속 버퍼에 모든 입력 데이터를 배치한다.

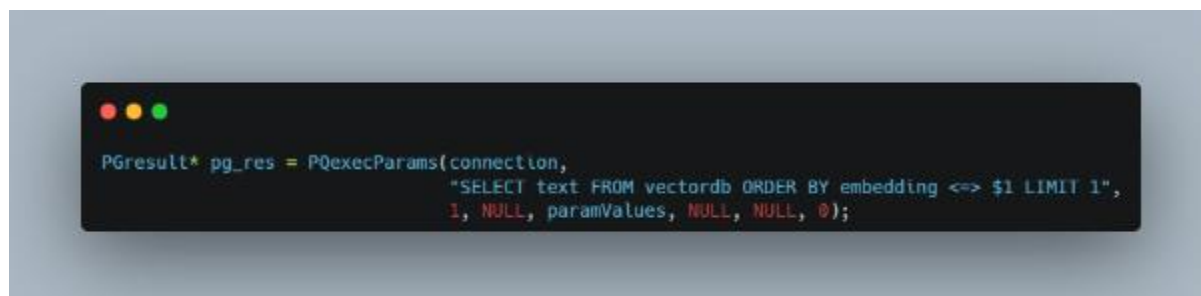
3.3.3. RAG 파이프라인 수행

모든 입력 읽기가 정상적으로 완료되면 `nvme_bdev_ctrlr_tokenize_calculate()`가 같은 callback 흐름에서 `do_rag()`를 호출하고, 이어서 토큰화를 수행한다. RAG 파이프라인은

다음 네 단계로 구성된다.

1) Embedding 생성. 읽어온 프롬프트에서 마지막 `<start_of_turn>user\n` 표시 뒤의 문자열을 사용자 질문으로 추출하고, HTTP를 통해 로컬 임베딩 서비스(`localhost:8080`)의 `/embedding` 엔드포인트로 전달한다. 요청 본문은 `content` 필드를 가진 JSON이다. 응답에서는 `[0]["embedding"]`에 있는 벡터를 선택한다. 제시된 설계·실험 자료의 임베딩 모델은 BGE-M3이지만, 호출 코드가 모델을 지정하지는 않으므로 실제 서비스의 모델과 실행 설정은 실험 환경에서 명시해야 한다.

2) Similarity Search. 생성된 embedding 벡터를 libpq를 사용하여 PostgreSQL의 벡터 데이터베이스에 전달하고, 코사인 거리 연산(`<=>`)을 기준으로 가장 유사한 문서를 검색한다.



Storage Server에서는 요청마다 DB 연결을 생성하고, 정상 처리 경로에서 결과 객체와 연결을 해제한다. 검색된 문서는 결과 객체를 해제하기 전에 C++ 문자열로 복사한다.

3) 프롬프트 구성(Query Generation). 기존 프롬프트의 이전 문맥, 검색된 참조 문서, 사용자 질문을 `[reference]`와 `[query]` 구분자를 사용하여 하나의 문자열로 결합한다.

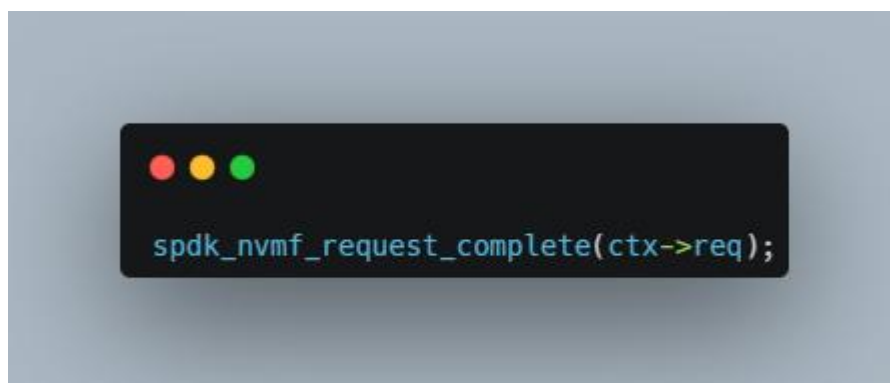
4) Tokenization. 결합된 문자열을 llama.cpp의 `llama_tokenize()` 함수를 이용하여 토큰 시퀀스로 변환한다. Storage Server는 `vocab_only`와 `no_alloc` 설정으로 모델을 로드하여 vocabulary만 참조하며, LLM 추론을 위한 전체 가중치를 로드하지 않는다. 이 모듈은 입력 토큰을 생성하는 역할만 담당하며, 답변을 생성하는 LLM 추론은 Edge Device에서 수행한다.

3.3.4. 토큰화 결과 기록과 요청 완료

토큰화가 정상적으로 완료되면 결과 버퍼의 앞 4 byte에 토큰 개수를, 이후 공간에 토큰

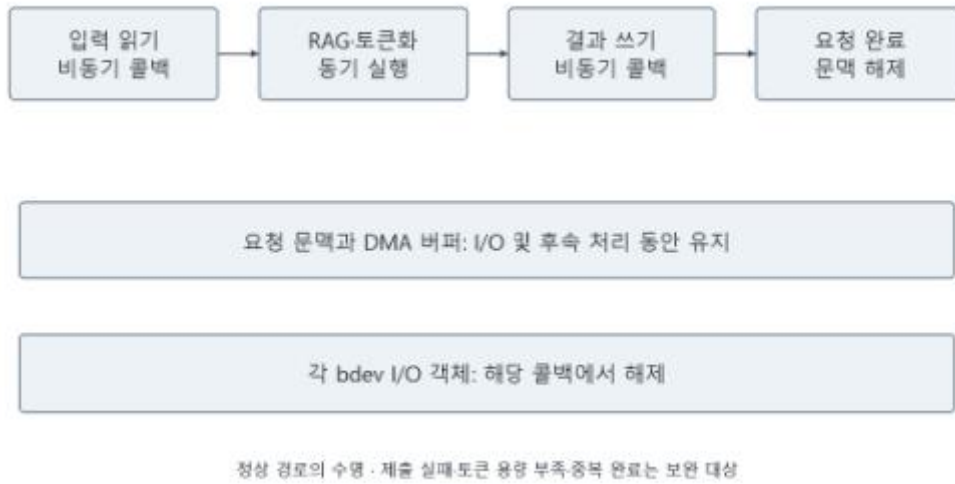
ID 배열을 기록한다. 이 결과를 `spdk_bdev_writev_blocks_ext()`를 통해 SPDK bdev 계층으로 `output.bin`이 위치한 NVMe 블록 영역에 직접 기록한다.

출력 extent가 여러 개인 경우 각 extent에 대해 순차적으로 write를 제출하며, 각 write의 완료는 callback으로 처리한다. 모든 write가 성공하면 `nvmf_bdev_ctrlr_tokenize_complete()`가 응답 상태를 성공으로 설정한다. 이어서 `bpe_tokenize_ctx_free()`가 메타데이터와 DMA 버퍼를 해제하고 아래 함수를 호출하여 NVMe-oF 요청을 완료한 뒤, 요청별 문맥을 해제한다. 개별 bdev I/O 객체는 해당 완료 callback에서 해제된다.



토큰 개수와 토큰 배열은 결과 파일에 기록되며, 명령 완료 응답 자체에 토큰 본문이나 별도 토큰 길이를 실는 구조는 아니다. 이 completion이 NVMe-oF를 통해 Edge Device에 도달하면, Edge는 3.2.4절에서 설명한 바와 같이 `output.bin`을 읽어 토큰 결과를 회수하고 LLM 추론을 수행한다.

아래는 Storage Server 내부의 정상 처리 순서와 요청 자원의 수명을 나타낸다.



Storage Server 내부의 I/O와 계산의 실행 문맥을 나타낸다. bdev를 통한 read와 write 제출은 비동기로 동작하지만, Embedding을 위한 HTTP 요청, DB 검색 및 토큰화는 read 완료 callback 내에서 동기적으로 실행된다.

본 구현은 파일의 물리적 저장 위치를 매개로 Storage Server의 RAG 파이프라인과 토큰화를 Edge Device의 LLM 추론 입력에 연결하는 처리 경로를 제공한다. Edge Device에는 입력·결과 파일 준비, extent 메타데이터 전송, 결과 파일 읽기 및 답변 추론이 남고, 검색·문서 결합·토큰화의 연산 부담은 Storage Server로 이동한다. 이 구조의 데이터 이동량 감소와 지연시간 변화에 대한 정량적 평가는 4장의 연구 결과 분석 및 평가에서 다룬다.

3.4. 인터페이스 적용 조건과 구현상의 제약

3.4.1. 주소 매핑과 결과 형식의 유지 조건

파일의 저장 구간을 직접 전달하는 구조에서는 FIEMAP의 물리 위치가 대상 bdev의 LBA와 대응해야 한다. 현재 코드는 물리 위치와 길이를 512로 나누지만, Storage는 명령의 블록 수에 자신의 bdev block 크기를 곱한다. 따라서 임의의 블록 크기나 파티션 구성에 그대로 적용할 수 있는 일반 주소 변환으로 해석해서는 안 된다. 또한 파일의 논리 오프셋은 전송하지 않으므로 sparse 파일의 빈 구간과 extent 순서·안정성을 확인해야 한다. 고정 파일명과 O_TRUNC를 사용하므로 여러 요청 또는 프로세스가 같은 파일을 동시에 사용하면 데이터가 섞일 수 있다.

입력·출력 메타데이터는 구조체의 메모리 표현을 복사하므로 자료형 폭과 byte 순서가 일치해야 한다. 수신 길이, 시작 offset, extent 개수와 크기 연산의 overflow는 구조체를 해석하기 전에 검사해야 하지만, 현재 구현에는 이러한 사전 검사가 부족하다. 결과 파일

의 토큰 개수 역시 실제 읽은 길이와 버퍼 용량 안에 있는지 확인해야 한다.

Storage의 vocabulary 및 BOS 등 특수 토큰 정책은 Edge의 추론 모델과 일치해야 한다. 기존 숫자 배열 입력 경로는 재토큰화를 피하지만, 문자열 입력을 요구하는 멀티모달 분기까지 호환성을 보장하지는 않는다. 임베딩 서비스의 응답 형식·벡터 차원과 DB 스키마도 양쪽의 실행 설정에서 확인해야 한다.

3.4.2. 오류 처리와 요청 수명

앞 절의 흐름은 정상 처리 경로에 대한 정적 분석 결과이다. 비동기 I/O의 제출 성공은 작업 완료를 뜻하지 않으며, 제출 함수가 음수로 실패하면 완료 callback이 오지 않는다는 API 계약에 맞추어 요청을 정리해야 한다. 현재 초기 read 제출 실패 경로에는 요청 완료가 중복될 수 있는 분기가 있고, 일부 제출 실패 및 토큰화 실패 경로에는 성공 상태로 완료되는 문제가 남아 있다. Edge의 명령 호출 검사도 음수 반환만 오류로 취급하므로 양수 NVMe 오류 상태를 별도로 처리해야 한다. 파일 읽기·쓰기의 짧은 반환 길이도 확인 대상이다.

HTTP 실패, 잘못된 JSON, SQL 오류와 검색 결과 0행, 모델 로드 실패 역시 정상 처리와 구분하여 검사해야 한다. Edge의 DB 연결은 재사용되지만 sleep 이후 복귀 과정의 재연결과 해제 후 포인터 관리에 보완이 필요하다. 이러한 조건을 이미 검증되거나 수정된 기능으로 취급하지 않는다. 비동기 read/write와 달리 HTTP·DB·토큰화는 SPDK callback 안에서 동기 실행되므로, 외부 서비스의 대기는 해당 SPDK thread의 다른 요청 처리에도 영향을 줄 수 있다.

3.4.3. 구현 의존성과 평가 범위

Edge에는 Linux FIEMAP, libnvme 및 libpq가 필요하고, Storage의 애플리케이션은 SPDK, libllama-ggml 및 libpq를 연결한다. 실제 헤더·라이브러리 버전과 C/C++ 빌드 설정, 모델·DB·장치 구성의 일치를 확인해야 한다. 소스에 호출 경로가 존재하는 것과 실제 환경에서 빌드·실행이 검증된 것은 구분한다.

CLI 작업의 RAG 처리 시간은 마이크로초 단위로 저장되어 최종 응답을 거쳐 콘솔에 전달된다. 이 구간에는 선택한 경로의 파일 I/O, 서비스 호출 및 입력 변환 등이 포함되므로, 답변 생성까지의 중단 지연이나 개별 타이머의 합계와 동일한 값으로 취급하지 않는다. Edge의 연결 재사용과 Storage의 요청별 DB 연결 차이도 측정 조건에 포함한다. 4장에서는 동일 질의·모델·DB·네트워크·캐시 조건과 원시 로그를 바탕으로 정확성, 실패 처리, 데이터 이동량 및 지연시간을 평가하며, 미확인 실험 수치를 임의로 채우지 않는다.

4. 연구 결과 분석 및 평가

본 연구에서는 NVMe-oF 기반 RAG 파이프라인 오프로딩 구조의 효과를 평가하기 위해 기존 방식과 제안 방식의 데이터 이동량 및 처리 지연을 비교하였다. 기존 방식은 Edge Device에서 RAG와 Tokenization을 수행하며 원격 Storage Server의 벡터 인덱스에 접근하는 구조이다. 제안 방식은 Embedding, Similarity Search 및 Tokenization을 Storage Server에서 수행하고, 생성된 토큰 시퀀스를 Edge Device로 반환하는 구조이다. 두 방식의 비교를 통해 데이터 이동 감소 효과를 확인하고, 단계별 지연 분석을 통해 성능 개선 요인을 평가하였다.

4.1. 실험 환경

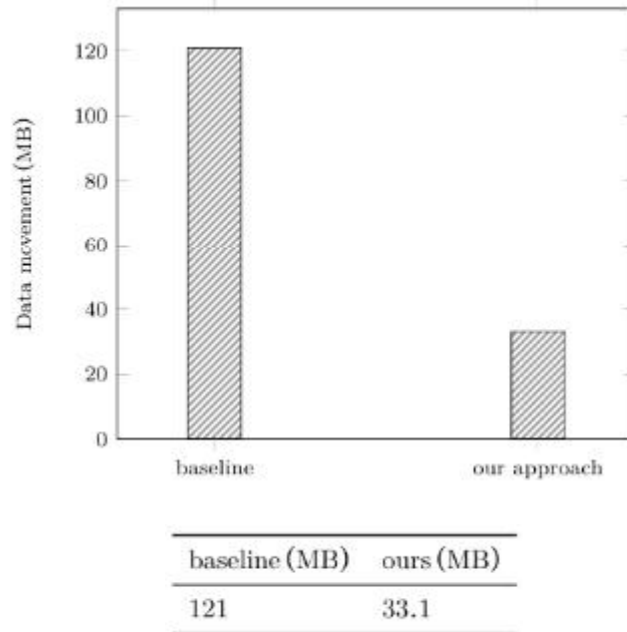
실험 시스템은 Jetson Orin Nano 기반 Edge Device와 SPDK 기반 Storage Server로 구성하였으며, 두 장치는 1 Gbps 네트워크에서 NVMe-oF/TCP로 연결하였다. Vector DB는 PostgreSQL과 pgvectorscale을 사용하여 구성하였고, 질의 임베딩에는 bge-m3 모델을 사용하였다. Edge Device의 LLM 추론은 llama.cpp 기반 Gemma 3 270M으로 수행하였다. 평가 워크로드는 TriviaQA를 사용하였으며, 데이터베이스의 규모는 약 7 GB이다. 주요 실험 환경은 다음과 같다.

Edge	Jetson Orin Nano
Vector DB	PostgreSQL (w/ pgvectorscale)
Embedding	bge-m3
LLM	Gemma 3 270M (llama.cpp)
워크로드	TriviaQA
데이터베이스	7 GB
네트워크 대역폭	1 Gbps

평가 지표는 Edge Device와 Storage Server 간 총 데이터 이동량, RAG 및 Tokenization의 단계별 처리 시간, Similarity Search 내부의 I/O 및 기타 처리 시간으로 구성하였다. 데이터 이동량은 워크로드 수행 중 해당 TCP 포트를 통해 송수신된 바이트를 누적하여 측정하였다.

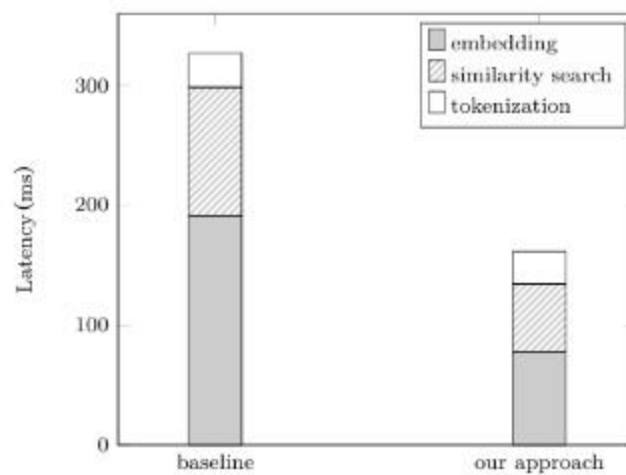
4.2. 실험 결과

4.2.1. 데이터 이동량 비교



워크로드 수행 중 Edge Device와 Storage Server 사이에서 발생한 총 데이터 이동량을 비교한 결과, 기존 방식은 121 MB, 제안 방식은 33.1 MB로 측정되었다. 제안 방식은 기존 방식 대비 87.9 MB의 데이터 이동을 줄였으며, 감소율은 약 72.6%이다.

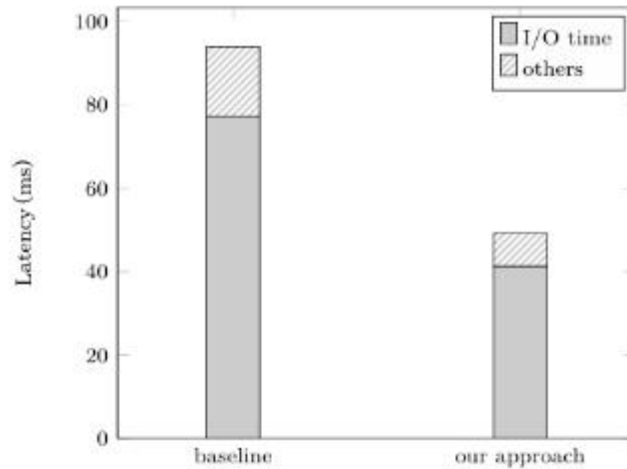
4.2.2. latency breakdown



	baseline (ms)	ours (ms)
embedding	191.13	77.18
vectordb	107.17	56.98
tokenize	28.40	27.20

오프로딩에 따른 처리 시간 변화를 확인하기 위해 Embedding, Similarity Search 및 Tokenization 단계의 지연을 비교하였다. Embedding 시간은 191.13 ms에서 77.18 ms로 감소하여 세 단계 중 가장 큰 시간 감소를 보였다. Similarity Search 시간은 107.17 ms에서 56.98 ms로 약 46.8% 감소하였다. Tokenization 시간은 28.40 ms에서 27.20 ms로 감소하였으나, 차이는 1.20 ms로 상대적으로 작았다.

4.2.3. similarity search breakdown



	baseline (ms)	ours (ms)
io time	77.180	41.272
others	16.786	7.990

I/O 시간은 77.180 ms에서 41.272 ms로 약 46.5% 감소하였으며, 기타 처리 시간은 16.786 ms에서 7.990 ms로 감소하였다.

5. 결론 및 향후 연구 방향

5.1. 결론

본 연구에서는 자원이 제한된 엣지 장치의 RAG 파이프라인과 토큰화를 스토리지 서버로 오프로딩하고, 처리된 토큰 시퀀스를 엣지에 반환하는 구조를 설계하고 구현하였다.

Jetson Orin Nano와 SPDK 기반 스토리지 서버를 이용하여 전용 하드웨어 없이 제안 구조를 검증하였으며, 기존 방식 대비 데이터 이동량을 약 73%, 유사도 검색 지연을 약 48% 감소시켰다. 이를 통해 데이터가 저장된 위치에서 검색과 토큰화를 수행하는 방식이 엣지 환경의 데이터 전송 부담과 원격 접근 오버헤드를 줄이는 데 효과적임을 확인하였다.

5.2. 향후 연구 방향

향후에는 검색 오프로딩과 토큰화 오프로딩의 효과를 분리하여, 각 기능이 데이터 이동량과 처리 지연 감소에 기여하는 정도를 정량적으로 분석할 필요가 있다. 또한 다양한 모델, 워크로드 및 코퍼스 규모에서 성능을 평가하여 제안 구조의 적용 범위와 한계를 검증하고자 한다.

6. 참고 문헌

[1] R. Mahapatra, H. Santhanam, C. Priebe, H. Xu, and H. Esmaeilzadeh, "In-Storage Acceleration of Retrieval Augmented Generation as a Service," Proc. of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), pp. 450-466, 2025.

[2] Gemma Team. (2025). Gemma 3 Technical Report [Online]. Available: <https://arxiv.org/abs/2503.19786> (downloaded 2026, May 31)

[3] ggml.ai. (2025). llama.cpp [Online]. Available: <https://github.com/ggml-org/llama.cpp> (downloaded 2026, May 31)

[4] Timescale. (2024). pgvectorscale: Complement pgvector with DiskANN-based Indexing [Online]. Available: <https://github.com/timescale/pgvectorscale> (downloaded 2026, May 27)

[5] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," Proc. of Findings of the Association for Computational Linguistics (ACL 2024), pp. 2318-2335, Aug. 2024.

[6] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension," Proc. of the 55th

Annual Meeting of the Association for Computational Linguistics (ACL 2017), pp. 1601-1611, Jul. 2017.

[7] E. Kim and S. Ahn, "Poster: Acceleration of RAG Pipeline with NVMe-oF based Near-Data Processing for Edge LLM Inference," Proc. of the IEEE International Conference on Non-Volatile Memory Systems and Applications (NVMSA 2026), 2026.

[8] E. Kim, Y. Yang, Y. Na, and S. Ahn, "RAG Pipeline Offloading via NVMe-oF for Edge LLM Inference," Proc. of the International Workshop on Computer Systems (IWCS 2026), 2026.